

ExamGuard AI — Online Examination Integrity & Proctoring Platform

1. Project Overview

ExamGuard AI is a MERN-based online examination platform with an independent Python computer-vision service. It is designed to detect and record potentially suspicious examination events such as missing face, multiple faces, phone/object presence, prolonged camera absence, and selected browser/session signals. The system produces explainable risk indicators for examiner review; it must not automatically declare a student guilty of cheating.

Core stack:

- Frontend: React + Vite + Tailwind CSS + Socket.IO client
- Backend: Node.js + Express + Socket.IO + JWT
- Database: MongoDB
- CV service: Python + FastAPI + OpenCV + YOLO
- Optional infrastructure: Redis + Docker
- Browser extension: Manifest V3, used only for explicitly disclosed and permitted exam-session signals.

2. Functional Requirements

1. Roles

- Student: authenticate, join an assigned exam, complete questions, grant required permissions, submit.
- Examiner: create exams, assign students, monitor sessions, review events, finalize review decisions.
- Admin: manage users, configuration, audit logs, model configuration and system health.

2. Core exam features

- Authentication and role-based authorization.
- Exam creation and editing.
- Question bank and randomized question ordering.
- Exam start/end rules and server-side timing.
- Autosave answers.
- Submission and result storage.
- Session lifecycle: CREATED, ACTIVE, SUBMITTED, EXPIRED, TERMINATED.

3. Proctoring signals

- FACE_MISSING
- MULTIPLE_FACES
- PHONE_DETECTED
- CAMERA_DISABLED
- CAMERA_BLOCKED
- EXCESSIVE_LOOKING_AWAY
- TAB_FOCUS_LOST (only where permitted and clearly disclosed)
- NETWORK_DISCONNECTED
- SUSPICIOUS_EVENT_BURST

4. AI/CV rules

- Sample frames rather than inferencing every camera frame.
- Use cheap OpenCV checks before expensive AI inference.
- Use temporal confirmation to reduce false positives.
- Deduplicate continuous events into intervals.
- Store confidence and evidence metadata.
- Never use a single detection as proof of cheating.

5. Risk engine

Each event contributes a configurable score with decay/cooldown rules.

Example starting weights:

PHONE_DETECTED=40

MULTIPLE_FACES=30
CAMERA_DISABLED=30
FACE_MISSING=15
EXCESSIVE_LOOKING_AWAY=10
TAB_FOCUS_LOST=10
NETWORK_DISCONNECTED=5
The final score is a review priority, not a guilt decision.

6. Non-functional requirements

- Event latency target: under 2 seconds for normal conditions.
- CV inference should degrade gracefully on low-end devices.
- API should remain stateless where possible.
- Sensitive video should not be retained by default.
- Audit all examiner/admin actions.
- Use HTTPS/WSS in production.

3. System Architecture

Browser/Extension -> React Exam UI -> Node/Express API

```
||
| +-> MongoDB
| +-> Redis (optional)
| +-> Socket.IO
|
+-> CV Gateway -> Python FastAPI -> OpenCV/YOLO
```

The browser should preferably perform lightweight preprocessing and only transmit event metadata or explicitly configured evidence. The CV service is isolated from the main Node process so models can be upgraded independently.

4. API Design

ExamGuard AI API v1

Auth

POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout

Exams

GET /api/v1/exams
POST /api/v1/exams
GET /api/v1/exams/:id
PATCH /api/v1/exams/:id
DELETE /api/v1/exams/:id
POST /api/v1/exams/:id/assign

Sessions

POST /api/v1/exams/:id/sessions
GET /api/v1/sessions/:id
POST /api/v1/sessions/:id/heartbeat
POST /api/v1/sessions/:id/submit

Proctoring

POST /api/v1/proctoring/events
GET /api/v1/sessions/:id/events
PATCH /api/v1/proctoring/events/:eventId/review

Dashboard
GET /api/v1/dashboard/overview
GET /api/v1/dashboard/students
GET /api/v1/dashboard/students/:studentId

CV service
POST /cv/v1/analyze/frame
POST /cv/v1/analyze/batch
GET /cv/v1/health

WebSocket events:
client -> server:
session:join
session:heartbeat
proctor:event

server -> examiner:
proctor:alert
session:status
risk:update

5. Dataset & Annotation Plan

Dataset Plan

Do not collect real student exam footage for the initial development dataset.
Start with public datasets and synthetic/consented recordings.

Classes for the first CV model:

- person
- phone
- face (if supported by the selected model)
- book
- laptop

For face-state logic, use a face detector/landmark model rather than creating a custom identity dataset.

Suggested data folders:
data/raw/
data/processed/
data/annotations/
data/splits/

Annotation format:
YOLO detection format:

Each image gets one .txt file with the same basename.

Recommended split:
70% train
20% validation
10% test

Quality rules:

- Include different lighting conditions.
- Include glasses and different face orientations.
- Include partial occlusion.
- Include empty scenes.
- Include multiple people.

- Include phones at different distances.
- Keep consent and licensing records for every custom recording.

6. Build Roadmap

Build Roadmap

Sprint 1: Repository, Docker Compose, MongoDB, Express, React, authentication.

Sprint 2: Exam CRUD, question bank, assignments, student exam session.

Sprint 3: Examiner dashboard, Socket.IO, session heartbeat.

Sprint 4: OpenCV face presence/count and event generation.

Sprint 5: YOLO phone/person detection and temporal confirmation.

Sprint 6: Risk engine, deduplication, event review workflow.

Sprint 7: Chrome extension and permitted browser/session signals.

Sprint 8: Testing, load testing, security hardening and deployment.

Definition of done for MVP:

- Student can take an exam.
- Examiner can monitor multiple sessions.
- CV service can generate at least FACE_MISSING, MULTIPLE_FACES and PHONE_DETECTED events.
- Events reach Node.js and the dashboard in real time.
- MongoDB stores sessions/events.
- Risk score is reproducible from configured rules.
- Examiner can review/dismiss an event.

7. Security & Privacy

Security & Privacy

- Use HTTPS/WSS in production.
- Hash passwords with Argon2 or bcrypt.
- Use short-lived access tokens and refresh-token rotation.
- Validate every API payload.
- Apply role-based authorization server-side.
- Rate-limit authentication and event endpoints.
- Never trust client-provided risk scores.
- Keep CV models and inference configuration server-controlled.
- Minimize retained camera data.
- Prefer event metadata over raw video retention.
- Encrypt sensitive data in transit and at rest where applicable.
- Maintain audit logs for examiner/admin actions.
- Clearly disclose what is monitored and obtain required consent.
- Provide a documented retention/deletion policy.
- Avoid using face recognition/identity matching unless there is a specific lawful requirement and appropriate governance.
- Do not use a single AI signal as proof of misconduct.

8. Initial Risk Rules

Event	Score	Cooldown	Description
PHONE_DETECTED	40	20	Phone/object detected and temporally confirmed
MULTIPLE_FACES	30	20	More than one face detected for the confirmation window
CAMERA_DISABLED	30	60	Required camera becomes unavailable
FACE_MISSING	15	15	No face for the configured persistence window
EXCESSIVE_LOOKING_AWAY	10	30	Repeated head-direction deviation
TAB_FOCUS_LOST	10	30	Browser focus lost where permitted

NETWORK_DISCONNECTED	5	60	Connectivity interruption
----------------------	---	----	---------------------------